



JC4003 Natural Language Processing

Lecture 4: Neural Network Language Models

Dr Xiao Li

xiao.li@abdn.ac.uk

Dr Ruizhe Li

ruizhe.li@abdn.ac.uk

Dr Siwei Liu

siwei.liu@abdn.ac.uk

Outline

Introduction to neural network

Recurrent Neural Networks (RNN, LSTM)

RNNs for NLP Tasks

- As Language Model
- Classification
- Generation

Attention mechanism and transformer

The training process for NLP models

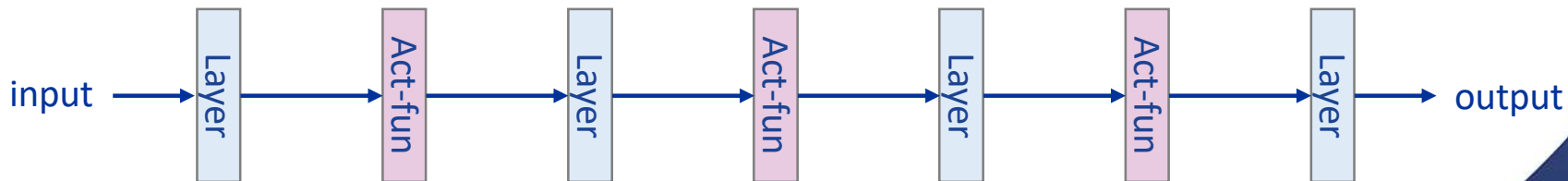
Introduction to Neural Networks

Neural Network Architecture

The most basic architecture of a neural network involves **stacking multiple layers**, which are **separated by activation functions**.

The simplest layer is the **dense layer**, which can be seen as a **matrix multiplication**. Different tasks may use other types of layers, such as LSTM, Transformer, convolutional layers, etc.

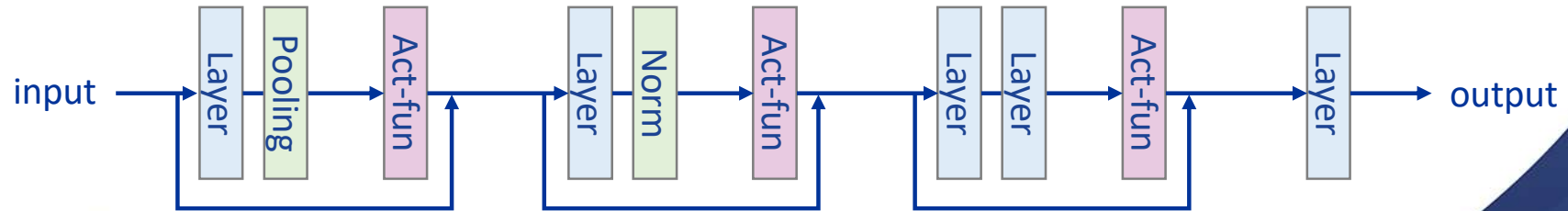
There are many **activation functions** to choose from, but modern neural networks typically adopt the **ReLU**, which is faster (rather than the traditional sigmoid).



Neural Network Architecture (cont.)

A neural network may also include other structures, such as normalization layers and pooling layers after each regular layer. There might also be residual connections between layers.

Whether it's activation functions, regular layers, or additional structures, all can be represented as simple mathematical operations.

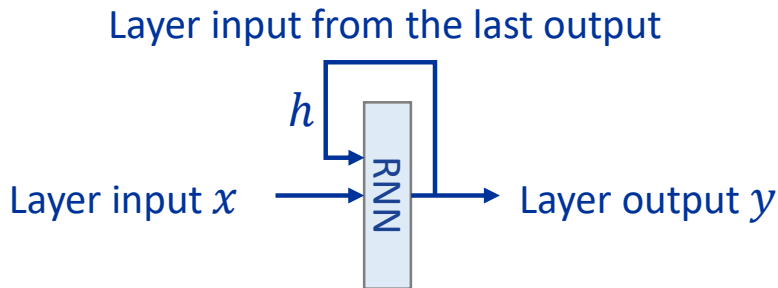


Recurrent Neural Networks

Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are a type of **neural network (or a layer of a neural network) designed to process sequences** by **maintaining a hidden state** that captures information from previous steps.

A RNN contains a **cycle within its network connections**, meaning that the value of some unit is directly, or indirectly, dependent on its own earlier outputs as an input. This gives RNNs a chance to access the historical inputs.



the next state

$$\langle h_i, y_i \rangle = RNN(x_i, h_{i-1})$$

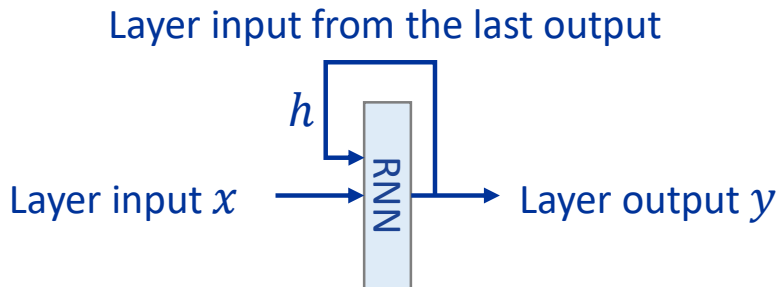
the previous state

Recurrent Neural Networks

The basic RNN layer implementation is simple, which can be defined by the following formulars:

$$h_i = g(W_1 h_{i-1}, W_2 x_i)$$
$$y_i = f(W_3 h_i)$$

where W_1, W_2, W_3 are matrices; g and f are active functions.



LSTM: Long Short-Term Memory

An advanced RNN layer is LSTM (Long Short-Term Memory), which is designed to capture long-term dependencies in sequential data by using gating mechanisms to regulate the flow of information. A LSTM use two hidden states h and c to capture the short and long historical information respectively.

$$f_t = \sigma_g(W_1x_t + W_2h_{t-1} + b_1)$$

$$i_t = \sigma_g(W_3x_t + W_4h_{t-1} + b_2)$$

$$o_t = \sigma_g(W_5x_t + W_6h_{t-1} + b_3)$$

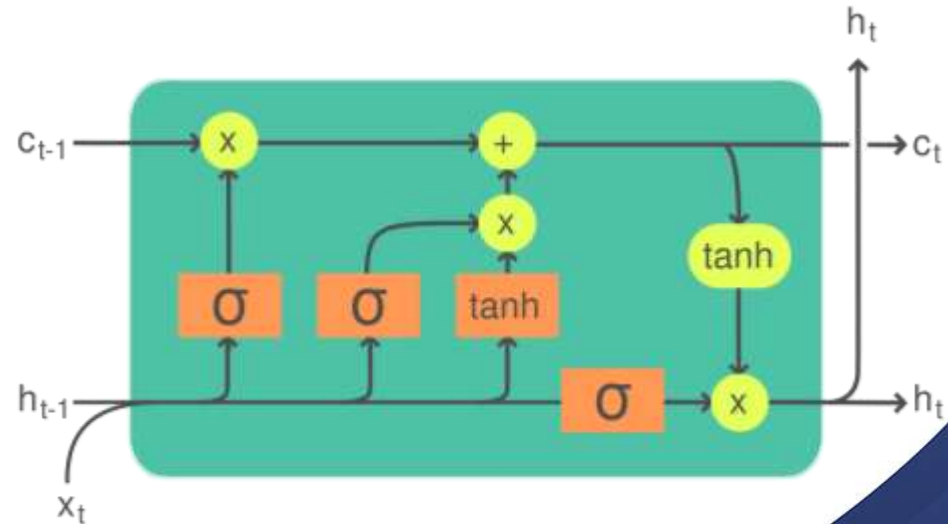
$$s_t = \sigma_c(W_7x_t + W_8h_{t-1} + b_4)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot s_t$$

$$h_t = o_t \odot \sigma_h(c_t)$$

where $\odot$ is the element-wise

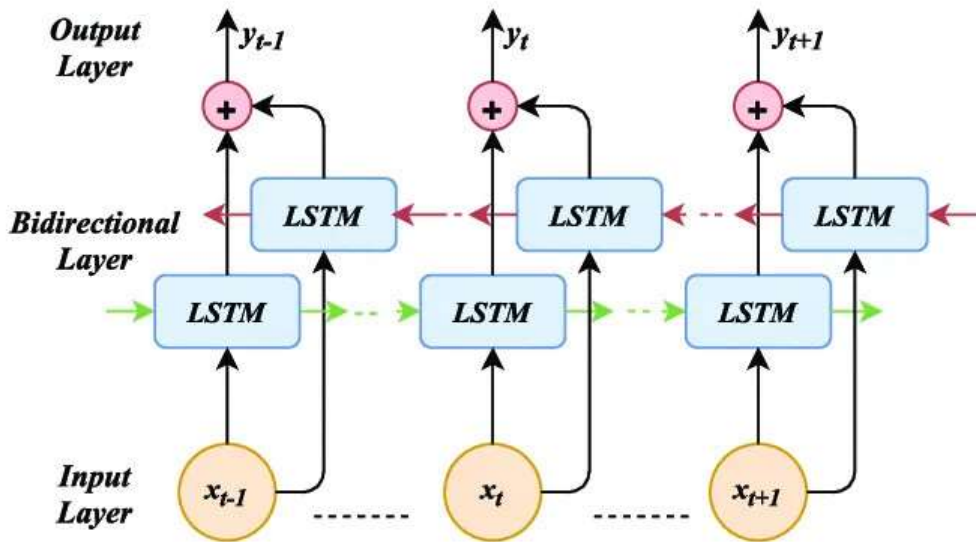
product



Combined RNN/LSTM Models

LSTM can be stacked and combined into more complex models. E.g.:

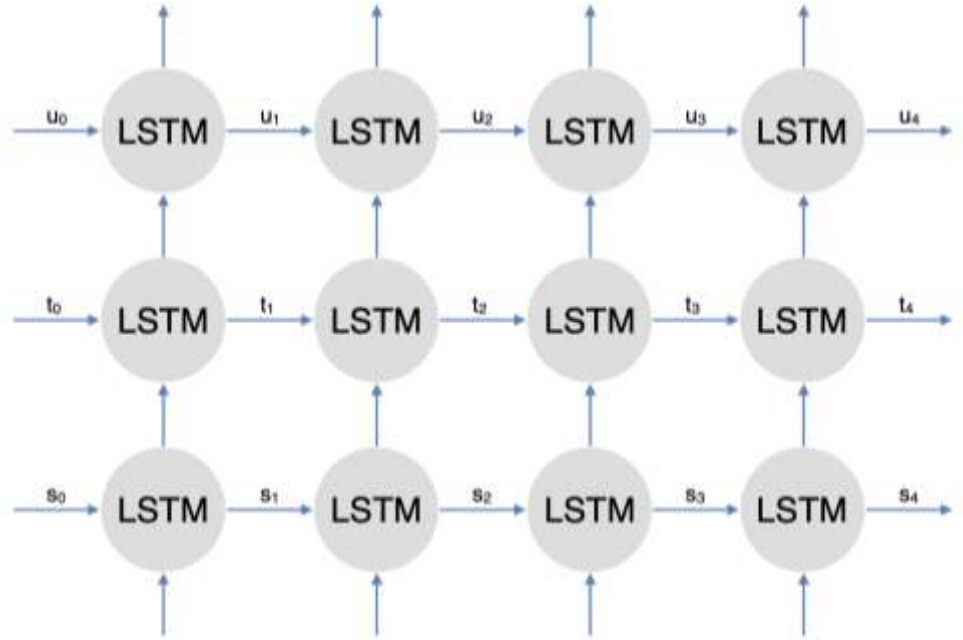
- **BiLSTM**: combining two LSTM layers that process the input sequence in both forward and backward directions, allowing the model to capture context from both past and future within the sequence.



Combined RNN/LSTM Models (cont.)

LSTM can be stacked and combined into more complex models. E.g.:

- **Multi-layer LSTM:** consists of multiple LSTM layers stacked on top of each other, where each layer passes its hidden state output to the next layer in the stack. The lower layer outputs are the higher layer inputs. This architecture allows the model to capture more complex patterns and hierarchical features in sequential data.



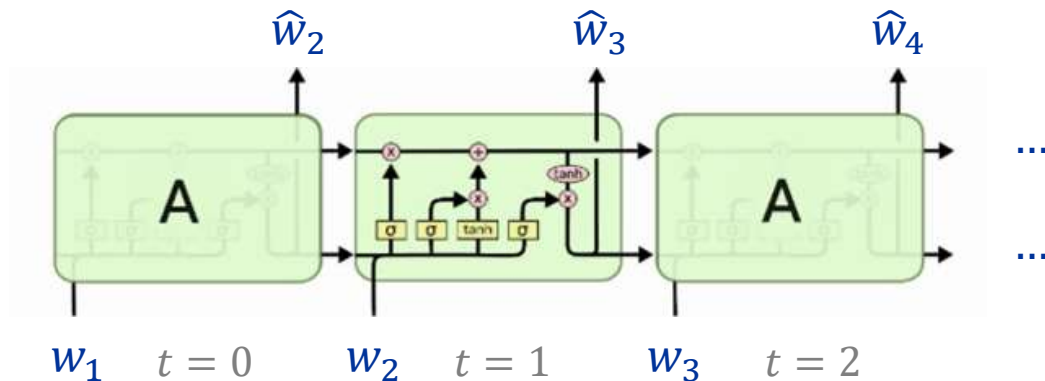
RNNs for NLP Tasks

- As Language Model

RNNs as Language Models

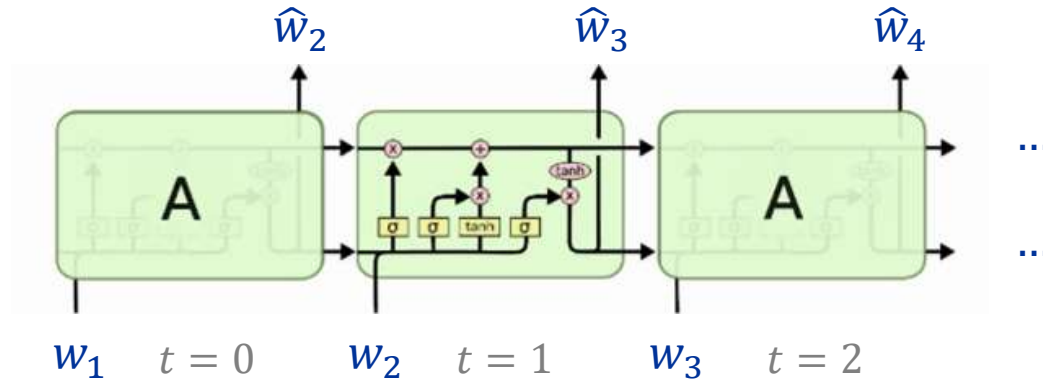
Recall that **language models predict the next word** in a sequence given some preceding context. RNN language models process the input sequence **one word at a time**, attempting to **predict the next word** from the current word and the previous hidden state.

This diagram shows how an LSTM works over time. The three cells represent the **same model** at **different time steps**, processing a sequence of inputs. In each time step $t = i$, the LSTM takes a word w_i , then, predicts the next word i.e. w_{i+1} .



RNNs as Language Models (cont.)

Thus, RNNs don't have the limited context problem that n-gram models have, or the fixed context that feedforward language models have, since the hidden state can in principle represent information about all of the preceding words all the way back to the beginning of the sequence.

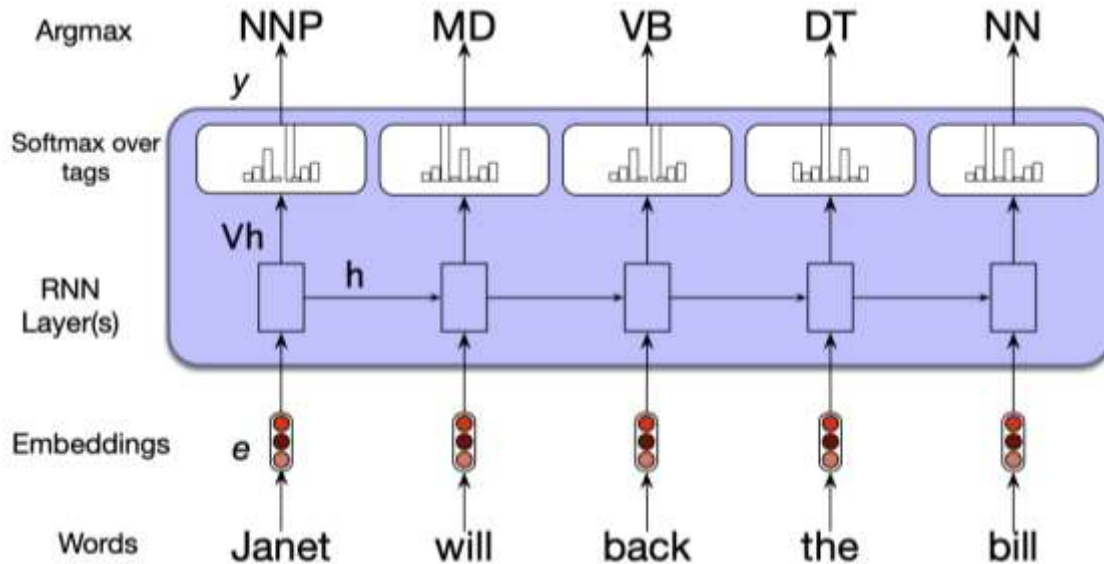


RNNs for NLP Tasks

- Classification

Sequence Labelling

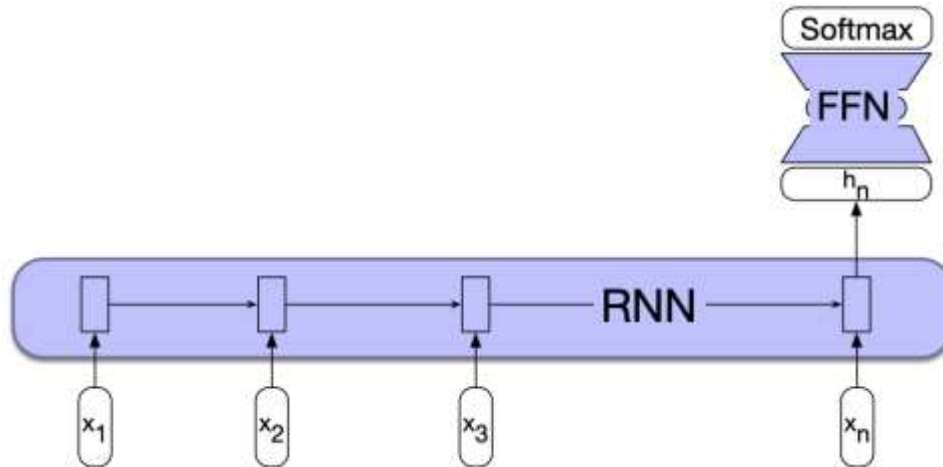
In sequence labelling, the network's task is to assign a label chosen from a small fixed set of labels to each element of a sequence. In an RNN approach to sequence labelling, inputs are word embeddings and the outputs are tag probabilities generated by a softmax layer over the given tagset.



Sequence Classification

Another use of RNNs is to **classify entire sequences** rather than the tokens within them. This is the set of tasks commonly called text classification, like sentiment analysis or spam detection.

To apply RNNs in this setting, we pass the text to be classified through the RNN a word at a time generating a new hidden vector at each time step. We can then take the **last hidden vector h_n** , to constitute a compressed representation of the entire sequence.



RNNs for NLP Tasks

- Generation

Sequence Generation

RNN-based language models can also be used for generation.

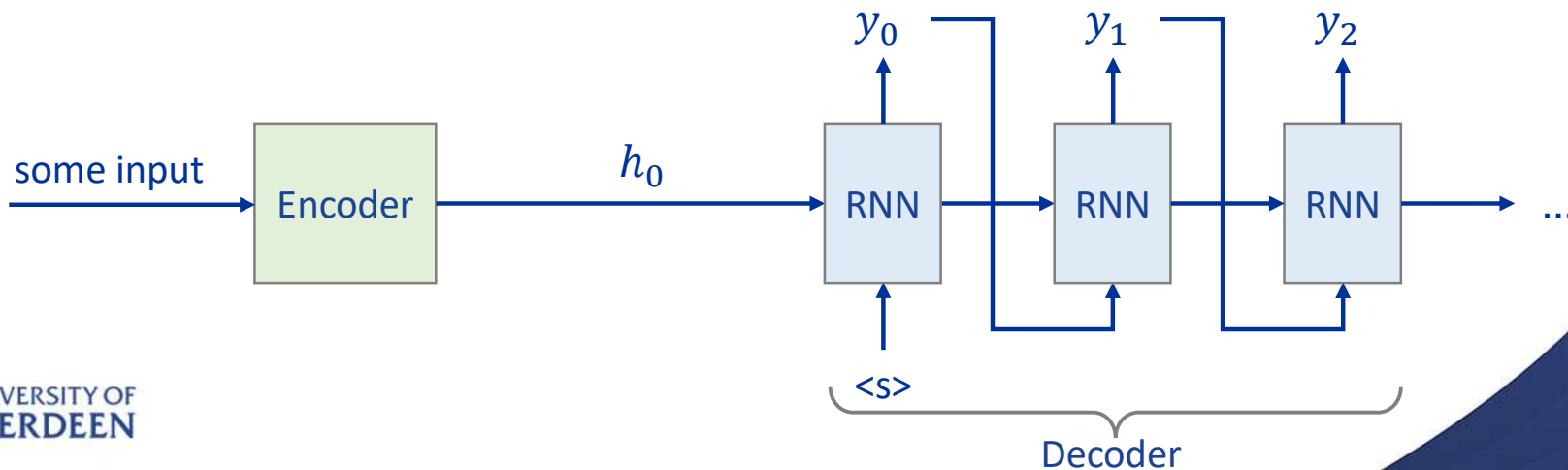
When generating sequences, an RNN typically relies on given information, which could be:

- a vector representation from an encoder's output
- directly from another sequence

This leads to different architectures for RNNs when used as generators.

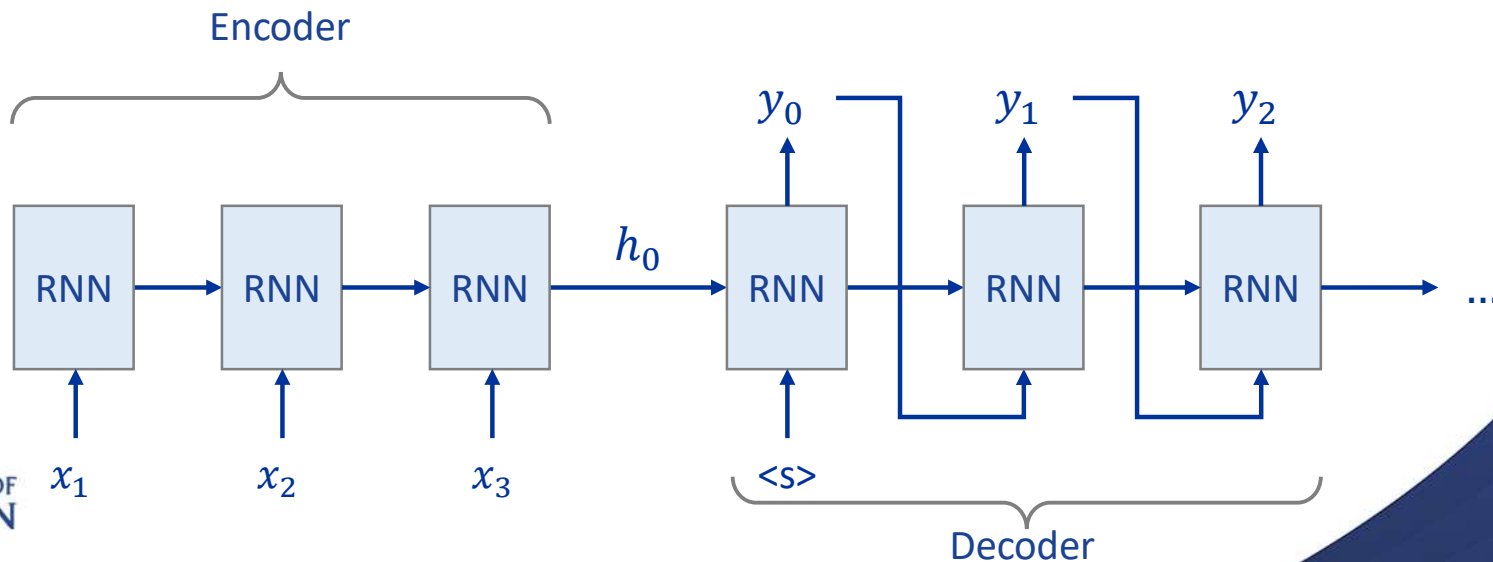
Encoder-Decoder Architecture

When an RNN generates a sequence based on the output of an encoder, the process typically follows the [encoder-decoder architecture](#). In this setup, the encoder processes the input sequence and compresses its information into a [fixed-length vector](#) (h_0), which the decoder RNN then uses to generate the output sequence, step by step.



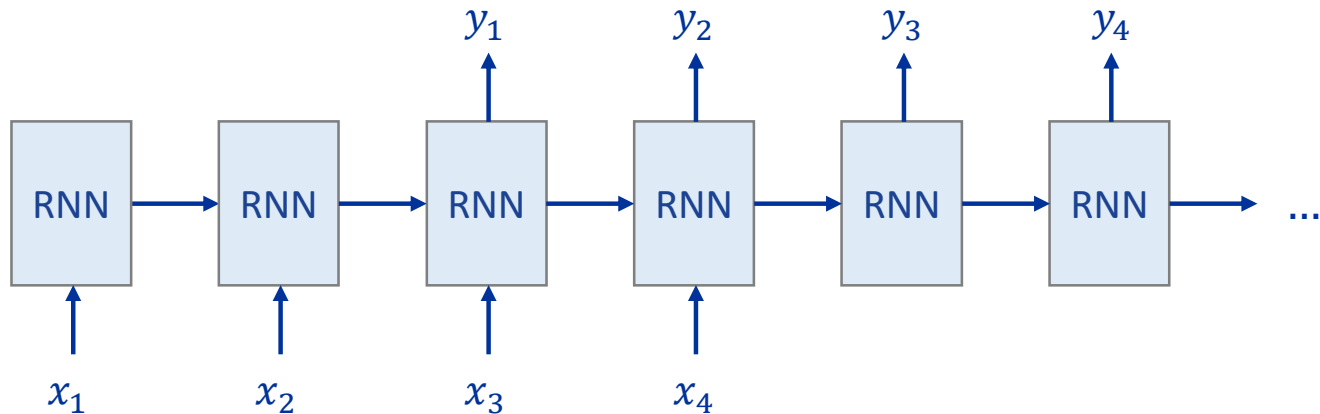
Encoder-Decoder Architecture

Here, the encoder can be [any neural network](#), as long as it can encode some form of information into a vector. It can also be an RNN, used to encode sequences such as text, e.g.:



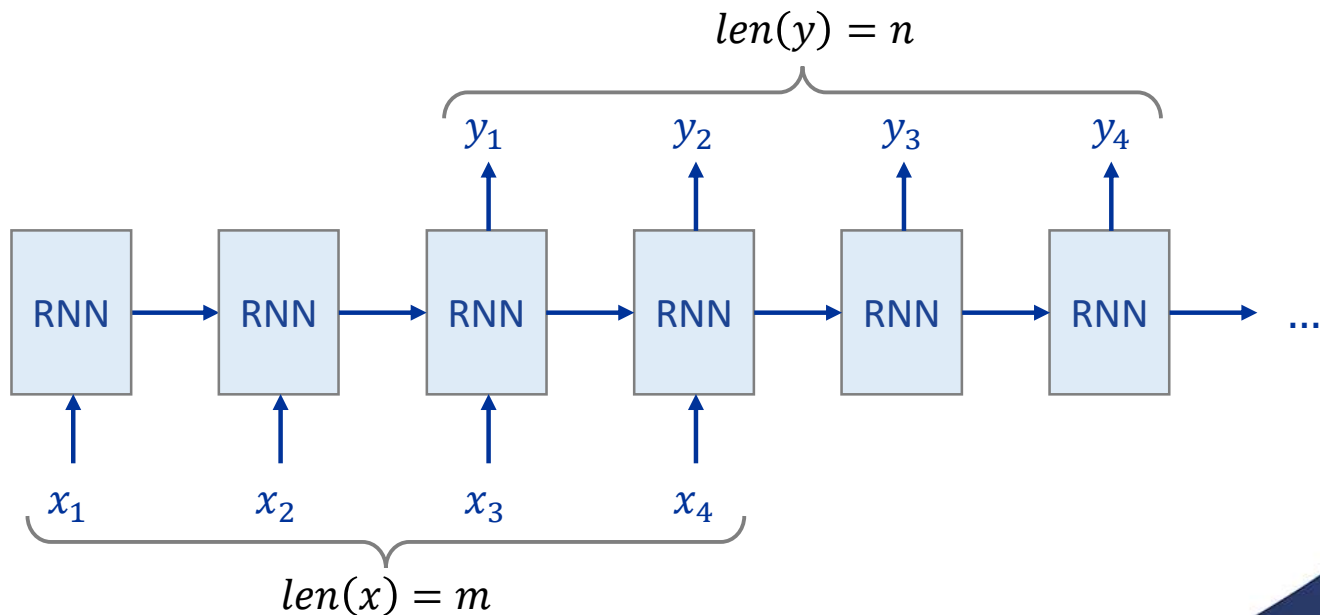
Many-to-Many Architecture

The Many-to-Many architecture in RNNs processes an input sequence where each input element generates a corresponding output element.



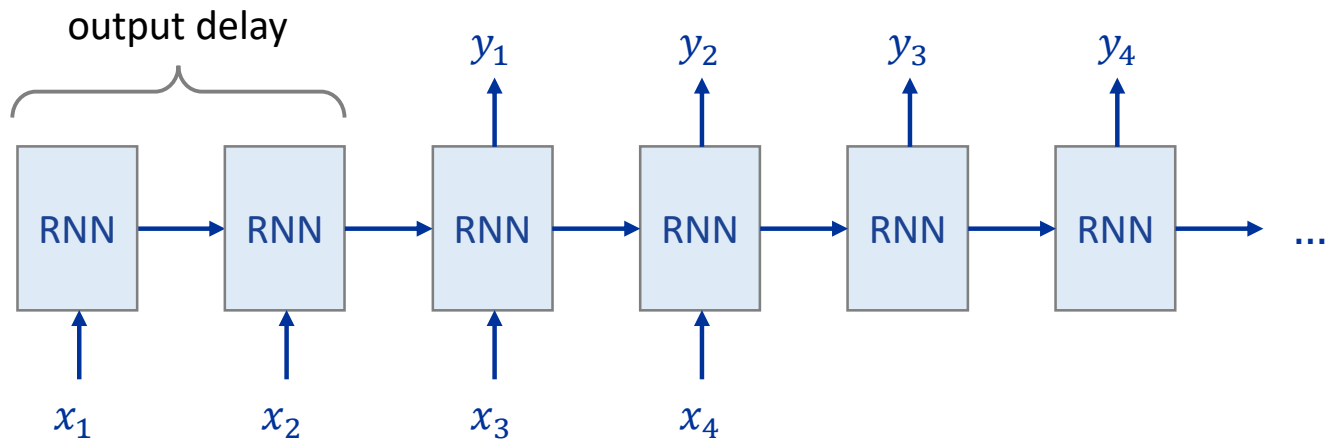
Many-to-Many Architecture (cont.)

Different from sequence labelling, in Many-to-Many RNN, the input and output sequences might **not be of the same length**.



Many-to-Many Architecture (cont.)

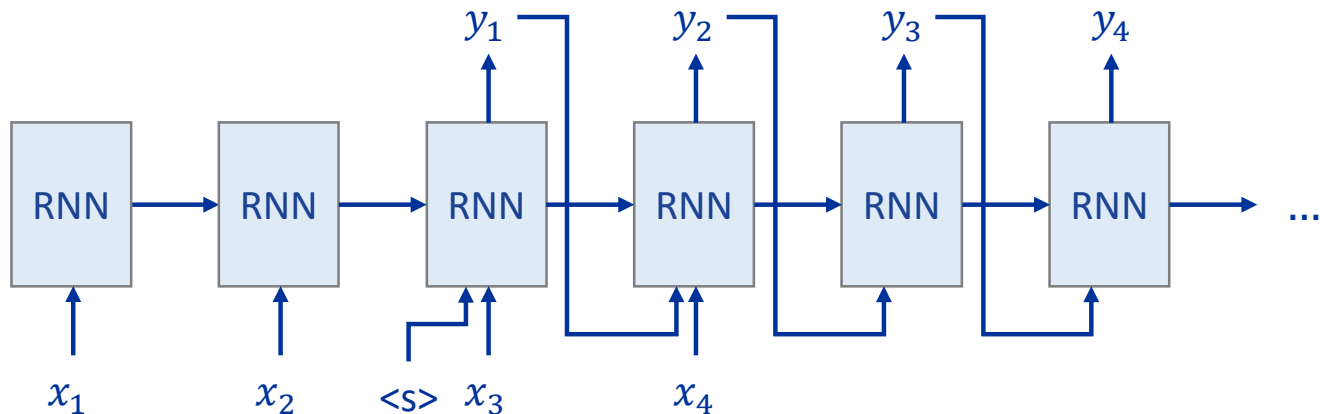
In a Many-to-Many architecture, the output often starts later than the input because the model needs to accumulate enough contextual information from the initial inputs before it can generate a meaningful output.



Many-to-Many Architecture (cont.)

In many cases, the Many-to-Many architecture also involves feeding the generated output from one time step back into the RNN as input for the next time step.

Similarly, $\langle s \rangle$ is passed to the RNN as the start marker of the output.



Conclusion

Attention Mechanism: Introduced attention layers, enhancing models by focusing on relevant input parts and handling long-range dependencies.

Transformer Model: Covered how the Transformer processes sequences in parallel using multi-head attention and feedforward networks, replacing traditional RNNs.

Training Models: Discussed the training process for NLP models, including classification and generative tasks, using techniques like teacher forcing and masking.

Position Embeddings: Addressed the importance of adding position embeddings to encode token order in the sequence.

Challenges in Transformer Training: Highlighted key challenges, such as weight initialization, learning rate scheduling, and regularization techniques to prevent overfitting.